LISTA DE EXERCÍCIOS 1

Dos Fundamentos da Linguagem até Arrays (Vetores e Matrizes)

Sumário

1 Fundamentos de C: Entrada/Saída, Tipos e Operadores **2**
1.1 Exercícios de Programação . . . 2
1.2 Questões de Múltipla Escolha . . . 2
1.3 Encontre o Erro . . . 4

2 Estruturas de Seleção: if, if/else e Operadores de Atribuição **5**
2.1 Exercícios de Programação . . . 5
2.2 Questões de Múltipla Escolha . . . 5
2.3 Encontre o Erro . . . 7

3 Estruturas de Repetição, switch e Operadores Lógicos **8**
3.1 Exercícios de Programação . . . 8
3.2 Questões de Múltipla Escolha . . . 8
3.3 Encontre o Erro . . . 10

4 Funções e Recursão **12**
4.1 Exercícios de Programação . . . 12
4.2 Questões de Múltipla Escolha . . . 12
4.3 Encontre o Erro . . . 14

5 Arrays (Vetores e Matrizes) **16**
5.1 Exercícios de Programação . . . 16
5.2 Questões de Múltipla Escolha . . . 17
5.3 Encontre o Erro . . . 19

1 Fundamentos de C: Entrada/Saída, Tipos e Operadores

1.1 Exercícios de Programação

1.1 - Calculadora de troco em cédulas e moedas

Escreva um programa que leia um valor a pagar e um valor entregue pelo cliente (ambos podendo ter centavos) e calcule o troco. Em seguida, decomponha o troco no menor número possível de cédulas e moedas, considerando as denominações do Real: R$100, R$50, R$20, R$10, R$5, R$2, R$1, R$0,50, R$0,25, R$0,10 e R$0,05. Utilize apenas operadores aritméticos (divisão inteira e módulo) - if e laços ainda não podem ser usados. Dica: trabalhe o troco em centavos (inteiro) para evitar erros de arredondamento em ponto flutuante.

1.2 - Precisão numérica: float vs. double

Declare a constante 1.0/3.0 como float e também como double, e imprima ambos os valores com 15 casas decimais usando printf. Em um comentário no código, explique por que os resultados diferem, relacionando a diferença com o número de bytes usado para armazenar cada tipo (consulte sizeof(float) e sizeof(double)).

1.3 - Overflow silencioso

Usando <limits.h>, declare duas variáveis int inicializadas com valores próximos de INT_MAX e some-as, imprimindo o resultado. O valor impresso está correto? Repita o experimento trocando o tipo para long long e depois para unsigned int, e comente, no código, as diferenças observadas em cada caso.

1.4 - Equação do 2º grau - parte 1 (sem estruturas de decisão)

Leia os coeficientes a, b e c de uma equação do 2º grau e calcule e imprima o discriminante ($\Delta = b^2 - 4ac$). Em seguida, calcule as duas raízes usando sqrt() de <math.h>, mesmo sem verificar o sinal de Δ. Execute o programa com um Δ negativo e explique, em comentário, o que a função sqrt() devolve nesse caso.

1.5 - Precedência e associatividade

Dada a expressão resultado = a + b * c - d / e % f + (g - h) * i; com a=2, b=3, c=4, d=20, e=6, f=4, g=10, h=3, i=2, calcule manualmente (no papel) o valor de resultado, indicando a ordem de avaliação de cada operador. Implemente o programa e confira sua resposta. Em seguida, insira parênteses em dois pontos da expressão de forma a alterar o resultado final, e explique o que mudou.

1.6 - Troca de valores sem variável auxiliar

Leia dois números inteiros e troque seus valores (swap) sem usar uma terceira variável, utilizando apenas operadores aritméticos (+ e −). Teste o programa com dois valores próximos de INT_MAX e explique, em comentário, por que essa técnica pode falhar nesse caso.

1.7 - Aritmética de caracteres

Leia um caractere maiúsculo com scanf("%c", ...) e, usando apenas operadores aritméticos sobre o char, imprima: (a) o código ASCII do caractere lido; (b) o próximo caractere do alfabeto; (c) a versão minúscula correspondente. Explique, relacionando com a tabela ASCII, por que essas operações funcionam.

1.2 Questões de Múltipla Escolha

Questão 1.1

Qual é a saída do trecho abaixo?

```c
int a = 7, b = 2;
printf("%d\n", a / b);
```

a) 3.5

b) 3

c) 4

d) 3.0

Questão 1.2

Considerando int x = 5; float y = 2; qual o resultado de x / y ?

a) 2

b) 2.0

c) 2.5

d) Erro de compilação

Questão 1.3

Qual especificador de conversão deve ser usado em scanf para ler corretamente um valor do tipo double?

a) %f

b) %lf

c) %d

d) %e - nunca funciona para double

Questão 1.4

Qual é o valor de x após a instrução abaixo?

```c
int x = 5;
x = x++ + ++x;
```

a) 11 (5 + 6)

b) 12 (6 + 6)

c) Comportamento indefinido - a linguagem C não garante um valor único

d) 10 (5 + 5)

Questão 1.5

Qual o valor lógico resultante da expressão abaixo, impresso por printf(“%d”, ...)?

```c
5 > 3 == 1
```

a) 1

b) 0

c) Erro de compilação

d) Depende do compilador

1.3 Encontre o Erro

Identifique o(s) problema(s) presentes nos códigos abaixo (podem ser erros de sintaxe, de lógica ou comportamento indefinido) e explique como corrigi-los.

Erro 1.1

```
int idade;
printf("Digite sua idade: ");
scanf("%d", idade);
printf("Voce tem %d anos.\n", idade);
```

Erro 1.2

```
float preco = 19.90;
int quantidade = 3;
float total;
total = preco * quantidade;
printf("Total: %d\n", total);
```

Erro 1.3

```
int a = 10, b = 3;
double resultado = a / b;
printf("Resultado: %.2f\n", resultado);
```

Erro 1.4

```
int x = 5, y = 10, soma;
soma = x + y
printf("Soma = %d\n", soma);
```

2 Estruturas de Seleção: if, if/else e Operadores de Atribuição

2.1 Exercícios de Programação

2.1 - Classificação de triângulos

Leia os comprimentos de três segmentos de reta (números reais positivos). Primeiro verifique, usando a desigualdade triangular, se eles podem formar um triângulo válido (a soma de quaisquer dois lados deve ser maior que o terceiro). Se formarem um triângulo válido, classifique-o quanto aos lados (equilátero, isósceles ou escaleno). Use apenas if/else aninhados.

2.2 - Ano bissexto

Leia um ano (inteiro) e determine, usando apenas if/else aninhados, se ele é bissexto. Lembre-se da regra completa: um ano é bissexto se for divisível por 4; porém, se também for divisível por 100, só é bissexto se for divisível por 400.

2.3 - Equação do 2º grau - parte 2 (com decisão)

Retome o exercício da equação do 2º grau do Tópico 1 e trate corretamente os três casos possíveis para o discriminante Δ: positivo (duas raízes reais distintas), igual a zero (uma raiz real dupla) ou negativo (nenhuma raiz real - apenas informe essa condição ao usuário, sem calcular números complexos).

2.4 - Menor entre três números, sem laços

Leia três números inteiros e determine, usando apenas if/else aninhados, qual deles é o menor. Em seguida, imprima os três números em ordem crescente (usando apenas as três variáveis lidas, sem vetores).

2.5 - Calculadora simples com validação

Leia dois números reais e um caractere representando uma operação (+, -, * ou /). Usando if/else if encadeados (switch ainda não foi visto), realize a operação correspondente. Trate o caso de divisão por zero (não realizando a divisão e exibindo uma mensagem de erro) e o caso de um operador inválido.

2.6 - Números de Armstrong de 3 dígitos

Um número de Armstrong de 3 dígitos é aquele em que a soma dos cubos de seus dígitos é igual ao próprio número (ex.: $153 = 1^3 + 5^3 + 3^3$). Leia um número inteiro de 3 dígitos e, extraindo cada dígito com divisão inteira e módulo (sem usar laços - extraia os 3 dígitos diretamente), verifique e informe se ele é um número de Armstrong.

2.7 - Pedra, papel e tesoura

Simule uma rodada do jogo "pedra, papel e tesoura" entre dois jogadores. Leia a jogada de cada jogador como um caractere ('r' para pedra/rocha, 'p' para papel e 't' para tesoura) e, usando if/else aninhados, determine e imprima o resultado: vitória do jogador 1, vitória do jogador 2 ou empate.

2.2 Questões de Múltipla Escolha

Questão 2.1

Qual é a saída do trecho abaixo para n = -5?

```
if (n > 0)
    if (n > 10)
        printf("Grande\n");
```

```
else
    printf("Nao positivo\n");
```

a) “Nao positivo”

b) “Grande”

c) Erro de compilação

d) Nenhuma saída é impressa

Questão 2.2

Qual é o valor final de x após as instruções abaixo?

```
int x = 10;
x -= 3;
x *= 2;
```

a) 7

b) 14

c) 17

d) 20

Questão 2.3

Qual é a saída do trecho abaixo?

```
int x = 5;
if (x = 0) {
    printf("Zero\n");
} else {
    printf("Nao zero\n");
}
```

a) “Zero”

b) “Nao zero”

c) Erro de compilação

d) Comportamento indefinido

Questão 2.4

Qual é a saída do trecho abaixo?

```
int i = 3;
int resultado = i++ * 2;
printf("i = %d, resultado = %d\n", i, resultado);
```

a) i = 4, resultado = 6

b) i = 4, resultado = 8

c) i = 3, resultado = 6

d) i = 3, resultado = 8

Questão 2.5

Qual é o valor de b impresso após as instruções abaixo?

```
int a = 5, b;
b = a > 3;
printf("%d\n", b);
```

a) 5

b) 3

c) 1

d) 0

2.3 Encontre o Erro

Erro 2.1

```
int nota = 5;
if (nota >= 7)
    printf("Aprovado\n");
    printf("Parabens!\n");
else
    printf("Reprovado\n");
```

Erro 2.2

```
float media = 0.1 + 0.2;
if (media == 0.3)
    printf("Media exata\n");
else
    printf("Media diferente de 0.3\n");
```

Erro 2.3

```
int nota;
printf("Digite a nota (0 a 10): ");
scanf("%d", &nota);
if (0 <= nota <= 10)
    printf("Nota valida\n");
else
    printf("Nota invalida\n");
```

Erro 2.4

```
int aprovado;
int nota = 8;
if (nota >= 7)
    aprovado = 1;
printf("Aprovado: %d\n", aprovado);
```

3 Estruturas de Repetição, switch e Operadores Lógicos

3.1 Exercícios de Programação

3.1 - Crivo de Eratóstenes (sem vetor)

Leia um número inteiro N ($N \geq 2$) e, usando laços for aninhados, imprima todos os números primos entre 2 e N. Para cada número do intervalo, use um laço interno para testar se ele possui algum divisor além de 1 e dele mesmo, interrompendo o teste com break assim que encontrar um divisor. Ainda não utilize vetores - eles serão vistos no próximo capítulo.

3.2 - Valor de π pela série de Leibniz

A série de Leibniz aproxima π por $\pi \approx 4 \times (1 - 1/3 + 1/5 - 1/7 + 1/9 - ...)$. Escreva um programa que use um laço do/while para somar termos da série até que o valor absoluto do termo atual seja menor que uma precisão informada pelo usuário (ex.: 0.0001). Imprima o valor aproximado de π obtido e o número de termos somados.

3.3 - Triângulo de Pascal (sem função fatorial)

Imprima as N primeiras linhas do Triângulo de Pascal (coeficientes binomiais). Utilize a propriedade de que cada elemento pode ser calculado a partir do anterior na mesma linha (coef = coef * (linha - coluna) / (coluna + 1);), evitando calcular fatoriais repetidamente. Use laços for aninhados.

3.4 - MDC e MMC pelo algoritmo de Euclides (iterativo)

Implemente, usando um laço while, o cálculo do máximo divisor comum (MDC) entre dois inteiros positivos pelo algoritmo de Euclides com resto (e não por subtrações sucessivas - em comentário, justifique por que a versão com módulo é mais eficiente). Em seguida, calcule o mínimo múltiplo comum (MMC) usando MMC(a,b) = (a × b) / MDC(a,b).

3.5 - Menu interativo com switch

Implemente uma calculadora com um menu (soma, subtração, multiplicação, divisão e sair), controlado por switch dentro de um laço do/while que se repete até o usuário escolher “sair”. Trate: (a) opções inválidas, usando default; (b) divisão por zero; (c) use break após cada case, exceto em um caso proposital em que dois cases compartilhem o mesmo tratamento por meio de fall-through intencional (comente essa decisão no código).

3.6 - Números perfeitos em um intervalo

Um número é perfeito quando é igual à soma de seus divisores próprios, excluindo ele mesmo (ex.: 6 = 1 + 2 + 3). Leia um intervalo [inicio, fim] e, usando laços aninhados, encontre e imprima todos os números perfeitos nesse intervalo, junto com seus divisores.

3.7 - Simulação de cara ou coroa

Usando rand() (com <stdlib.h>) e srand(time(NULL)) (com <time.h>) para inicializar a semente, simule N lançamentos de moeda. Conte, com um laço for, quantas vezes saiu cara e quantas saiu coroa, e imprima os percentuais com duas casas decimais.

3.2 Questões de Múltipla Escolha

Questão 3.1

Qual é o valor impresso pelo trecho abaixo?

```
int i;
for (i = 0; i < 5; i++)
```

```
    ;
printf("%d\n", i);
```

a) 4

b) 5

c) Laço infinito

d) Erro de compilação

Questão 3.2

Qual é a saída do trecho abaixo?

```
int x = 10;
do {
    printf("%d ", x);
    x++;
} while (x < 5);
```

a) Nada é impresso

b) “10 ”

c) Laço infinito

d) Erro de compilação

Questão 3.3

Para n = 2, qual é a saída do trecho abaixo?

```
switch (n) {
    case 1: printf("Um ");
    case 2: printf("Dois ");
    case 3: printf("Tres "); break;
    default: printf("Outro ");
}
```

a) “Dois ”

b) “Dois Tres ”

c) “Um Dois Tres ”

d) “Outro ”

Questão 3.4

Qual é a saída do trecho abaixo?

```
int x = 0;
if (x != 0 && 10 / x > 1)
    printf("A\n");
else
    printf("B\n");
```

a) “A”

b) “B”

c) Erro em tempo de execução (divisão por zero)

d) Erro de compilação

Questão 3.5

Qual é a saída do trecho abaixo?

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1)
            break;
        printf("%d%d ", i, j);
    }
}
```

a) 00 01 02 10 11 12 20 21 22

b) 00 10 20

c) 00 01 10 11 20 21

d) Laço infinito

3.3 Encontre o Erro

Erro 3.1

```
int i = 0;
while (i < 10) {
    printf("%d\n", i);
}
```

Erro 3.2

```
int soma = 0;
for (int i = 1; i <= 10; i++) {
    soma += i;
}
printf("Soma dos 9 primeiros numeros: %d\n", soma);
```

Erro 3.3

```
int dia = 3;
switch (dia) {
    case 1: printf("Segunda\n"); break;
    case 2: printf("Terca\n"); break;
    case 3: printf("Quarta\n");
    case 4: printf("Quinta\n"); break;
    default: printf("Dia invalido\n");
}
```

Erro 3.4

```
int soma = 0;
for (int i = 1; i <= 5; i++);
{
    soma += i;
}
printf("Soma = %d\n", soma);
```

4 Funções e Recursão

4.1 Exercícios de Programação

4.1 - Potência e fatorial: recursivo vs. iterativo

Implemente duas funções recursivas: long long potencia(int base, int expoente) (para expoente $\geq$ 0) e long long fatorial(int n). Implemente também versões iterativas dessas mesmas funções (com nomes diferentes) e, no main(), compare os resultados das duas versões para os mesmos valores de entrada, garantindo que sejam idênticos.

4.2 - MDC e MMC recursivos e composição de funções

Implemente a função recursiva `int mdc(int a, int b)` pelo algoritmo de Euclides (mdc(a,b) = mdc(b, a % b), com caso base mdc(a,0) = a). Implemente `int mmc(int a, int b)`, que chama internamente sua função mdc. No main(), leia pares de números até que o usuário digite 0 0, imprimindo o mdc e o mmc de cada par.

4.3 - Conversão de base numérica com recursão

Escreva a função recursiva `void converteBase(int numero, int base)` que imprime a representação de numero na base especificada (2, 8 ou 16 - para hexadecimal, trate dígitos de 10 a 15 como 'A' a 'F'). Dica: a chamada recursiva com numero / base deve ocorrer ANTES do `printf` de numero % base, para que a pilha de chamadas inverta corretamente a ordem dos dígitos.

4.4 - Contagem de primos em um intervalo, com composição de funções

Implemente `int ehPrimo(int n)`, que retorna 1 se n for primo e 0 caso contrário. Usando essa função (sem duplicar a lógica), implemente `int contaPrimos(int inicio, int fim)`, que retorna quantos primos existem no intervalo. No main(), demonstre que alterar uma cópia do parâmetro dentro de ehPrimo não afeta a variável original passada pelo chamador (evidenciando a passagem por valor).

4.5 - Contador persistente com variável static

Escreva `void chamarContador(void)`, com uma variável local static int chamadas = 0; que é incrementada e impressa a cada chamada. Escreva também `void chamarSemStatic(void)`, idêntica, mas com a variável automática (sem static). Chame cada função 5 vezes em um laço no main() e explique, em comentário, a diferença observada.

4.6 - Escopo: variável global vs. local com o mesmo nome

Declare uma variável global int contador = 100;. Escreva `void testaEscopo(void)`, que declara uma variável LOCAL de mesmo nome (contador), inicializada com 0, a incrementa e a imprime. Chame a função três vezes no main() e, ao final, imprima o contador global. Explique, em comentário, por que os valores são diferentes (sombreamento de variável).

4.2 Questões de Múltipla Escolha

Questão 4.1

Qual é a saída do programa abaixo?

```
void dobrar(int x) {
    x = x * 2;
}

int main(void) {
    int num = 5;
    dobrar(num);
```

```
    printf("%d\n", num);
    return 0;
}
```

a) 5

b) 10

c) 0

d) Erro de compilação

Questão 4.2

Qual é a saída do programa abaixo?

```
void conta(void) {
    static int c = 0;
    c++;
    printf("%d ", c);
}

int main(void) {
    conta(); conta(); conta();
    return 0;
}
```

a) 1 1 1

b) 0 1 2

c) 1 2 3

d) 3 3 3

Questão 4.3

O que ocorre ao chamar soma(5) no trecho abaixo?

```
int soma(int n) {
    return n + soma(n - 1);
}
```

a) Retorna 15 corretamente

b) Retorna 0

c) Estouro de pilha (stack overflow), pois falta um caso base para encerrar a recursão

d) Erro de compilação

Questão 4.4

Sem um protótipo de quadrado antes do main, o que ocorre ao compilar o código abaixo em C99/C11?

```
int main(void) {
    int r = quadrado(4);
    printf("%d\n", r);
```

```
    return 0;
}

int quadrado(int x) {
    return x * x;
}
```

a) Compila e executa normalmente, imprimindo 16

b) Erro de compilação (ou, no mínimo, um aviso severo de "implicit declaration"), pois quadrado é usada antes de ser declarada

c) Apenas um aviso sem consequência - o programa sempre funciona normalmente

d) Comportamento indefinido apenas em tempo de execução, mas compila sem qualquer aviso

Questão 4.5

Qual é a saída do programa abaixo?

```
int x = 10;

void func(void) {
    int x = 20;
    printf("%d ", x);
}

int main(void) {
    func();
    printf("%d\n", x);
    return 0;
}
```

a) 20 20

b) 10 10

c) 20 10

d) 10 20

4.3 Encontre o Erro

Erro 4.1

```
int soma(int a, int b) {
    int resultado = a + b;
}

int main(void) {
    printf("%d\n", soma(3, 4));
    return 0;
}
```

Erro 4.2

```
int fatorial(int n) {
    return n * fatorial(n - 1);
```

```
}
```

Erro 4.3

```c
void incrementar(int valor) {
    valor = valor + 1;
}

int main(void) {
    int x = 5;
    incrementar(x);
    printf("x = %d\n", x);
    return 0;
}
```

Erro 4.4

```c
int soma(int a, int b);

int main(void) {
    double resultado = soma(3.5, 2.7);
    printf("%.2f\n", resultado);
    return 0;
}

int soma(int a, int b) {
    return a + b;
}
```

5 Arrays (Vetores e Matrizes)

5.1 Exercícios de Programação

5.1 - Bubble sort instrumentado

Implemente o bubble sort (ordenação por flutuação) para ordenar um vetor de N inteiros em ordem crescente. A cada troca realizada, imprima o estado completo do vetor. Ao final, informe o total de comparações e de trocas realizadas. Adicione a otimização clássica: se, em uma passada completa, nenhuma troca ocorrer, o vetor já está ordenado e o algoritmo deve parar antes das N−1 passadas (use uma flag booleana).

5.2 - Busca binária vs. busca sequencial

Implemente int buscaSequencial(int v[], int n, int chave) e int buscaBinaria(int v[], int n, int chave) (esta última assumindo o vetor já ordenado). Ambas devem contar o número de comparações realizadas até encontrar (ou não) o elemento. Gere um vetor ordenado de 1000 elementos e compare o número de comparações das duas buscas para localizar o último elemento do vetor.

5.3 - Média, mediana e moda sem bibliotecas prontas

Leia N números inteiros para um vetor. Calcule e imprima: (a) a média aritmética; (b) a mediana (implementando você mesmo um algoritmo de ordenação antes de calculá-la); (c) a moda (valor mais frequente, com sua frequência). Não utilize nenhuma função pronta de ordenação de bibliotecas externas.

5.4 - Jogo da velha com verificação automática de vencedor

Represente um tabuleiro de jogo da velha com char tabuleiro[3][3], inicializado com espaços em branco. Simule as jogadas de dois jogadores ('X' e 'O'), alternando turnos, lendo linha e coluna de cada jogada e validando se a posição está livre. Após cada jogada, verifique automaticamente (linhas, colunas e as duas diagonais) se há um vencedor, ou se o jogo empatou (tabuleiro cheio sem vencedor).

5.5 - Multiplicação de matrizes

Leia as dimensões e os elementos de duas matrizes A (m×n) e B (n×p), verificando que o número de colunas de A é igual ao número de linhas de B. Calcule e imprima, formatada em linhas e colunas, a matriz produto C = A × B (m×p).

5.6 - Remoção de elementos duplicados

Leia N números inteiros para um vetor. Construa um segundo vetor contendo apenas os valores distintos do primeiro, preservando a ordem da primeira ocorrência de cada valor, sem usar um vetor auxiliar maior do que o necessário. Imprima o vetor resultante e quantos elementos duplicados foram removidos.

5.7 - Vetores paralelos: boletim de turma

Leia, para N alunos, o código de matrícula (inteiro) em um vetor matriculas[] e a nota final em um vetor paralelo notas[]. Encontre e imprima: matrícula e nota do aluno com a maior nota; matrícula e nota do aluno com a menor nota; e a média da turma. Trate o caso de empate na maior nota, listando todos os alunos empatados.

5.8 - Crivo de Eratóstenes com vetor de flags

Reimplemente o exercício de números primos do Tópico 3, agora usando um vetor crivo[N+1] como marcador: para cada primo p encontrado, marque todos os seus múltiplos como "não primos" (crivo[multiplo] = 0), evitando testar divisibilidade repetidamente para cada número. Em um comentário, explique por que essa abordagem é mais eficiente para valores grandes de N.

5.9 - Matriz simétrica

Leia uma matriz quadrada N×N de inteiros e verifique se ela é simétrica, isto é, se M[i][j] == M[j][i] para todo par (i, j). Caso não seja simétrica, informe todas as posições (i, j) em que a simetria falha.

5.10 - Rotação de vetor in-place

Dado um vetor de N inteiros e um valor K, rotacione o vetor K posições para a direita (o elemento que "sai" pela direita reaparece à esquerda). Implemente a rotação usando apenas um vetor auxiliar de tamanho K. Desafio extra: implemente a versão totalmente in-place, sem vetor auxiliar algum, usando o algoritmo de inversão em três etapas (inverter os primeiros N−K elementos, inverter os últimos K elementos e depois inverter o vetor inteiro).

5.11 - Elemento mínimo em um array

Escreva uma função recursiva que encontre e retorne o elemento mínimo em um array, onde o array e seu tamanho são dados como parâmetros.

```
//return the minimum element in a[]
int findmin(int a[], int n)
```

5.12 Maximizar lucro

Crie um programa em C que receba um array de preços em que precos[i] é o preço de uma determinada ação no i-ésimo dia. Você deseja maximizar seu lucro escolhendo um único dia para comprar uma ação e escolher um dia diferente no futuro para vender essa ação. Retorne o lucro máximo que você pode obter com esta transação. Se você não conseguir nenhum lucro, retorne 0.

```
Exemplo:

Entrada: preços = [7,1,5,3,6,4]
Saída: 5
```

Explicação: Compre no dia 2 (preço = 1) e venda no dia 5 (preço = 6), lucro = 6-1 = 5.

Observe que comprar no dia 2 e vender no dia 1 não é permitido porque você deve comprar antes de vender.

5.2 Questões de Múltipla Escolha

Questão 5.1

O que ocorre ao executar o trecho abaixo?

```c
int v[5] = {10, 20, 30, 40, 50};
printf("%d\n", v[5]);
```

a) Imprime 0

b) Erro de compilação

c) Comportamento indefinido - C não verifica limites de vetor em tempo de execução

d) Imprime 50 novamente

Questão 5.2

Qual é a saída do programa abaixo?

```
void zera(int v[], int n) {
    for (int i = 0; i < n; i++)
        v[i] = 0;
}

int main(void) {
    int a[3] = {1, 2, 3};
    zera(a, 3);
    printf("%d %d %d\n", a[0], a[1], a[2]);
    return 0;
}
```

a) 1 2 3 (o vetor original não é alterado, pois C usa passagem por valor)

b) 0 0 0

c) Erro de compilação

d) Comportamento indefinido

Questão 5.3

Qual é a saída do trecho abaixo?

```
int v[5] = {1, 2};
printf("%d %d %d\n", v[2], v[3], v[4]);
```

a) Lixo de memória em todas as posições

b) 0 0 0

c) 1 1 1

d) Erro de compilação

Questão 5.4

Qual é a saída do trecho abaixo?

```
int v[10];
int tamanho = sizeof(v) / sizeof(v[0]);
printf("%d\n", tamanho);
```

a) 10

b) 40 (o número de bytes)

c) 1

d) Erro de compilação

Questão 5.5

Qual assinatura de função é válida para receber uma matriz declarada como int m[3][4]?

a) void f(int m[][], int linhas, int colunas)

b) void f(int m[][4], int linhas)

c) void f(int *m[], int linhas, int colunas)

d) void f(int m, int linhas, int colunas)

5.3 Encontre o Erro

Erro 5.1

```
int v[10];
for (int i = 0; i <= 10; i++) {
    v[i] = i * 2;
}
```

Erro 5.2

```
int v[5];
int soma = 0;
for (int i = 0; i < 5; i++) {
    soma += v[i];
}
printf("Soma = %d\n", soma);
```

Erro 5.3

```
void imprime(int m[][], int linhas, int colunas) {
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++)
            printf("%d ", m[i][j]);
        printf("\n");
    }
}
```

Erro 5.4

```
int a[3] = {1, 2, 3};
int b[3] = {1, 2, 3};

if (a == b)
    printf("Vetores iguais\n");
else
    printf("Vetores diferentes\n");
```